

Group Assignment 3
Software Now HIT137

Name & ID

Mohammad Nahid Hossain - S401546

Fahim Munshi - S399955

Asif Hossain Hemel- S401807

Faisal Hossain - S403084

Introduction

This Project is mainly to overview the understanding and demonstrate the difference between all the three tools which are python, openCV and Tkinter. This application helps users to find the image and it has some challenges like modifying the images from the different identification tools.

Concept of this project such as

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Class interaction
- Constructors and methods

The program consists of three large modules written in Python:

The image_processor.py file is used for image processing and automatic difference generation.

2. game_logic.py – Contains code for the game state, score, and determining when a player clicks on an object.

3. gui_app.py – GUI application with Tkinter.

It is a combination of image processing with an interactive GUI which creates an interesting and informative game.

Project Objectives To use the concepts of Object Oriented Programming in a real application.

To manipulate images using OpenCV.

- To develop GUI application using Tkinter.

To handle the user and game logic effectively.

To illustrate interaction among classes and modular programming.

Technologies Used

The project relies on a number of Python libraries and frameworks:

Technology Purpose

Python Main programming language

OpenCV (cv2) Image processing and manipulation

NumPy_Numerical operations and array operations.

Tkinter GUI creation

Random Random difference generation

System Overview

The program functions in the following order:

1. The user loads an image.
2. Class ImageProcessor generates a copy of the image with changed values.

Five random modifications to the image are made.

4. The GUI shows the original and altered images.

5. Player selects differences in the altered image.

The DifferenceGame class determines if the click was correct or not.

7. GUI updates the score, mistakes and game status.

The application will terminate when:

- All five differences are found, or
makes a copy of them and adds some random visual variations.

This module demonstrates:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

The conditions surrounding the alteration of an abstract base class.

The Alteration class is an abstract base class that is the parent of all image modification classes.

class Alteration(ABC):

Features

- Stores minimum and maximum region sizes.
- Defines random regions in the area of the image.
- Defines the method apply, which is abstract.

@abstractmethod

There are several subclasses of Alteration:

- ColourShift
- BlurPatch
- BrightnessChange
- HueRotation
- NoiseInjection

In each subclass, apply() is implemented differently.

Example:

class BlurPatch(Alteration):

image Alteration Types

1. ColourShift

Adjusts the 'red' channel up, and the 'blue' channel down, to produce a perceptible difference in colour.

2. BlurPatch

Performs the Gaussian blur on a portion of the image.

```
cv2.GaussianBlur(region, (21, 21), 0)
```

3. BrightnessChange

Makes a part of an image darker.

4. HueRotation

Modifies the hue of the colour in HSV colour space.

5. NoiseInjection

Applies salt and pepper noise to a region.

ImageProcessor Class

The ImageProcessor class handles all the image operations.

```
class ImageProcessor:
```

Responsibilities

Read an image from a file. Load image from disk.

- Resize large images

Click on the "clone" icon to duplicate the image.

Make 5 distinct differences that are not the same.

- Store difference regions

Encapsulation

The class has private attributes:

```
self._original
```

```
self._modified
```

```
self._difference_regions
```

The variables are properly accessed through property methods

Random Difference Generation

Both alterations and positions are selected at random in the program.

```
random.shuffle(alteration_pool)
```

This makes each game new and different.

Preventing Overlap

This enhances the experience of the game as all the differences are obvious.

This module should include the following: This module should contain the following:

Purpose

The game_logic.py module handles all the game state.

Responsibilities include:

- Tracking found differences
- Counting mistakes
- Determining game-over conditions
- Managing cumulative score

DifferenceGame Class

```
class DifferenceGame:
```

This class maintains all information about this game.

Encapsulation

Game data are protected using private variables.

```
self._differences
```

```
self._mistakes
```

```
self._game_active
```

The internal state cannot be altered by the user.

Constructor

The constructor sets up the game variables.

```
def __init__(self):
```

It sets:

- Total differences found

- Total images played
- Mistake count
- Game status

Starting a New Round

def start_round(self, difference_regions):

This method:

- Resets game state
- Loads new difference regions
- Activates the game

This enhances the experience of the game as all the differences are obvious.

This module should include the following: This module should contain the following:

Purpose

The game_logic.py module handles all the game state.

Responsibilities include:

- Tracking found differences
- Counting mistakes
- Determining game-over conditions
- Managing cumulative score

DifferenceGame Class

class DifferenceGame:

This class maintains all information about this game.

Encapsulation

Game data are protected using private variables.

`self._differences`

`self._mistakes`

`self._game_active`

The internal state cannot be altered by the user.

Constructor

The constructor sets up the game variables.

```
def __init__(self):
```

It sets:

- Total differences found
- Total images played
- Mistake count
- Game status

Starting a New Round

```
def start_round(self, difference_regions):
```

This method:

- Resets game state
- Loads new difference regions
- Activates the game

Click Detection

The `check_click()` method tests the clicks of the players.

```
def check_click(self, click_x, click_y):
```

The method:

- Computes distance from the point that was clicked

- Identifies if the click is correct or not
- Revises number of points and errors

Distance Formula

The game determines the distance from a click to the difference region.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d = 16.97 \quad \Delta x = 12 \quad \Delta y = 12$$

A mathematical formula to find out if the player was close enough to a difference that was hidden.

Mistake Handling

Players can make a total of 3 errors in the game.

MAX_MISTAKES = 3

If the user is over this limit:

- The game ends
- The player loses the round

Reveal Feature

The reveal_all() method displays all the differences that were hidden.

```
def reveal_all(self):
```

This feature allows users to understand where the difference(s) were missed.

Properties

There are read-only access properties to the class via property decorators.

Example:

```
@property
```

```
def mistakes(self):
```

This makes it more secure and keeps it encapsulated.

Module 3: GUI Application (gui_app.py)

Purpose

gui_app.py: This module is responsible for the graphical interface of the game with the help of Tkinter.

The GUI takes on the responsibility to:

- Displaying images
- Handling user interaction
- Updating status labels

Making circles around differences found

GameApp Class

```
class GameApp(tk.Tk):
```

This class is an extension of the class tk.Tk, demonstrating inheritance.

GUI Layout

The interface contains:

- Header
- Toolbar
- Status bar
- Two image canvases

The left image is the original image and the right image is the modified image.

Buttons

Load Image Button

Enables user to choose an image file.

```
filedialog.askopenfilename()
```

Reveal All Button

Shows all differences that are not visible.

Canvas Interaction

The mouse clicks are detected by the modified image canvas.

```
self._canvas_mod.bind("<Button-1>", self._on_canvas_click)
```

Interactivity is generated by this event-driven programming.

Image Rendering

Images are converted to be compatible with Tkinter from OpenCV images.

```
cv2.cvtColor(cv2_img, cv2.COLOR_BGR2RGB)
```

It's converted with Pillow (PIL).

Drawing Difference Circles

If any difference is found by the player, then the program draws a coloured circle.

```
canvas.create_oval()
```

Red circles represent identified differences, blue circles represent discovered differences.

Status Updates

The GUI is constantly updated:

- Remaining differences
- Mistake count
- Total score
- Current game status

This enhances user experience and feedback.

OOP Concepts Used:

1. Encapsulation

Encapsulation protects data by enclosing it in private variables.

Examples:

- `_mistakes`
- `_differences`
- `_original`

Access is controlled via getter methods and properties.

2. Inheritance

Multiple instances of inheritance.

Examples:

```
class BlurPatch(Alteration)
```

```
class GameApp(tk.Tk)
```

Subclasses will inherit behaviour from their parent classes.

3. Polymorphism

`Apply()` is overridden in different ways by each alteration subclass.

remove the `apply` method from the image object.

This enables other interfaces to execute more than one behavior.

4. Abstraction

The abstract base class Alteration is used to mask the implementation and to ensure structure.

@abstractmethod

5. Class Interaction

The GUI communicates with:

- ImageProcessor
- DifferenceGame

This modular structure has the advantage of maintaining and reading it.

Strengths of the Project

User-Friendly Interface

The GUI is both looks and works nice.

Random Gameplay

Every game is different due to the random differences.

Modular Design

Using modules takes the program apart to make it easier to organize.

Strong OOP Implementation

The project provides good examples of some of the key concepts in OOP.

Real-World Application

Integrates computer vision with interactive games.

Limitations

Although the project works successfully, there are some limitations:

Limited Difference Types

There are only five types of alteration implemented.

No Timer

The game does not include a countdown timer.

Single Player Only

There is no support for multiplayer.

No Difficulty Levels

The level of the game does not change.

Future Improvements

There are several enhancements that can be made to the project:

Add Timer System

Introduce time-based gameplay.

Add Difficulty Levels

Give the user the options of easy, medium or hard mode.

Multiplayer Mode

Compete and play against others.

Save Scores

Save high scores on files and/or databases.

More Image Effects

Add more advanced OpenCV transformations.

Sound Effects

Include sound effects for clicking and game over.

Conclusion

The Spot the Difference Game is a successful example that shows how a complete application can be created using Python that integrates object-oriented programming, image processing and graphical user interface development.

The project effectively applies important OOP concepts including:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

The ImageProcessor module is used to modify images with OpenCV, the DifferenceGame module is used to control the game logic and the GameApp module is used to build an interactive Tkinter GUI.

Project overall, it's both educational and entertaining and demonstrates practical Python software development skills.